Can we achieve multiple inheritance using default method :

```
interface Alpha                         interface Beta
{                                       {
  default void m1()                       default void m1()
  {                                       {
  }                                       }
}                                       }
         class Implementer implements Alpha, Beta
         {
            public Implementer()
            {
              super();
            }

            //Here compulsory we need to override default method otherwise
            //ambiguity issue

         }
```

if both the default methods are having same name then overriding is compulsory

Multiple inheritance is possible in java by using default methods also inside an interface, here we need to use super keyword to differenciate the super interface methods.

Before java 1.8, we have abstract method inside an interface but now we can write method body(default method) so, to execute the default method inside an interface we need to take super keyword with interface name(Alpha.super.m1()).

**Whenever a class is implementing from two OR more than two super interafecs and If the interfaces are containing SAME default method name then we have an ambiguity issue so, compulsory we need to override default method.

//Program

```java
package com.ravi.default_method;

interface Alpha
{
    default void m1()
    {
        IO.println("Alpha interface default method m1");
    }
}

interface Beta
{
    default void m1()
    {
        IO.println("Beta interface default method m1");
    }
}

class Implementer implements Alpha, Beta
{
    @Override
    public void m1()
    {
        Alpha.super.m1();
        Beta.super.m1();
        IO.println("Overriding is compulsory due to same method name");
        IO.println("MI is possible");
    }
}

public class MultipleInheritanceUsingDefault
{
    public static void main(String[] args)
    {
        Implementer i = new Implementer();
        i.m1();
    }

}
```

Note : We can observe from above program that if two default methods are having same name then overriding is compulsory otherwise compilation error will be generated.

Same rule is applicable for fields also so variable hiding is compulsory as shown in the program below

```java
package com.ravi.default_method;

interface A
{
    int x = 100;
}

abstract class B
{
    int x = 200;
}

class C extends B implements A
{
    int x = 300; //Variable Hiding is compulsory to resolve ambiguity issue

    public void show()
    {
        IO.println(x);
    }
}
public class VariableHiding {

    public static void main(String[] args)
    {
        new C().show();
    }

}
```

What is static method of an interface :

We can write static method (method with body) inside an interface from java 8 version.

Example :

```
public interface Calculator
{
  static double getCube(int num) //java 8 [Access Modifier is public]
  {
  }
}
```

A static method must be marked with the static keyword and contains method body.

A static method without any access modifier is assumed to be public.

A static method cannot be marked as abstract OR final.

A static method is not inherited and cannot be accessed in a class OR interface directly, **It can be accessible through interface name only.**

Program that describes static method of an interface is by default public so we can access from another package.

```java
package com.ravi.interface_static_method;

public interface Calculator
{
    static  double doSum(double x, double y)  //JDK 1.8 [AM is public]
    {
           return x + y;
    }

    static double getCube(int num)  //JDK 1.8   [AM is public]
    {
         return num*num*num;
    }
}


package com.ravi.use;

import com.ravi.interface_static_method.Calculator;

public class Main
{
    public static void main(String[] args)
    {
        double sum = Calculator.doSum(12, 90);
        IO.println("Sum is :"+sum);

        double cube = Calculator.getCube(5);
        IO.println("Cube is :"+cube);
    }

}
```

WAP to show that static method of an interface is available to same interface only, It is not available to class OR interafce.

//Program

```
interface A
{
     static void m1()
     {
          IO.println("Static method");
     }
}
class B implements A
{
     //m1() static method is not available here
}

public class StaticMethodDemo
{
     public static void main(String[] args)
     {
          A.m1(); //interface static method we can access through interafce name only
     }
}
```

//Program

```
interface A
{
     static void m1()
     {
          IO.println("Static method");
     }
}
interface B extends A
{
     //m1() static method is not available here
}

public class StaticMethodDemo1
{
     public static void main(String[] args)
     {
          B.m1(); //Compilation error
     }
}
```

WAP to show we can write main method inside an interafce and it will be executed.

//Main.java

```
public interface Main
{
     public static void main(String[] args) //JDK 1.8
     {
          System.out.println("Hello World! through interface ");
     }
}
```

Introduction to Functional Programming :

----

* It is new and very popular feature which is introduced from JDK 1.8V.

* It is used to enable Functionla Programming in java.

* The main purpose of Functionla Programming to reduce the length of functions/methods by using Lambda expression.

What is a Functional Interface ?

----

* If an interface contains **exactly one abstract method then It is called Functional interface.**

* A **Functional interface** may contain 'n' number of default and static methods but it must contain only one abtract method.

* In order to restrict the developer to accept only one abstract method, Java has introduced @FunctionalInterface annotation

* @FunctionalInterface annoation is optional.

Example :

```
@FunctionalInterface
interface A
{
     void accept(); //SAM [Single Abstract Method]

     default void m2()
     {
     }

     static void m3()
     {
     }
}
```

//Program:

```
@FunctionalInterface
interface A
{
     void accept(); //SAM [Single Abstract Method]

     default void m2()
     {}

     static void m3()
     {}
}

public class StaticMethodDemo
{
     public static void main(String[] args)
     {
          A a1 = new A()
          {
               @Override
               public void accept()
               {
                    IO.println("Accept method is overriden using Anonymous inner class");
               }
          };

          a1.accept();
     }
}
```